



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Text Representation

Natural Language Processing

<http://vku.udn.vn/>

1



Contents

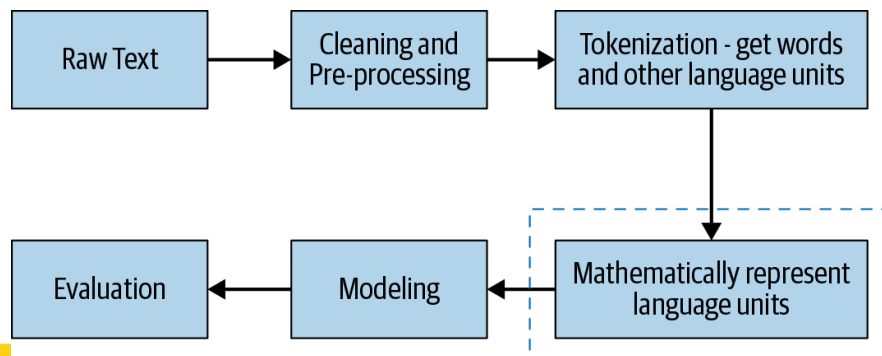
- Vector Space Models
- Basic Vectorization Approaches
 - One-Hot Encoding
 - Bag of Words
 - Bag of N-Grams
 - TF-IDF
- Distributed Representations
 - Word Embedding
 - CBOW
 - SkipGram
- Visualizing Embeddings

<http://vku.udn.vn/>

2

VKU Introduction

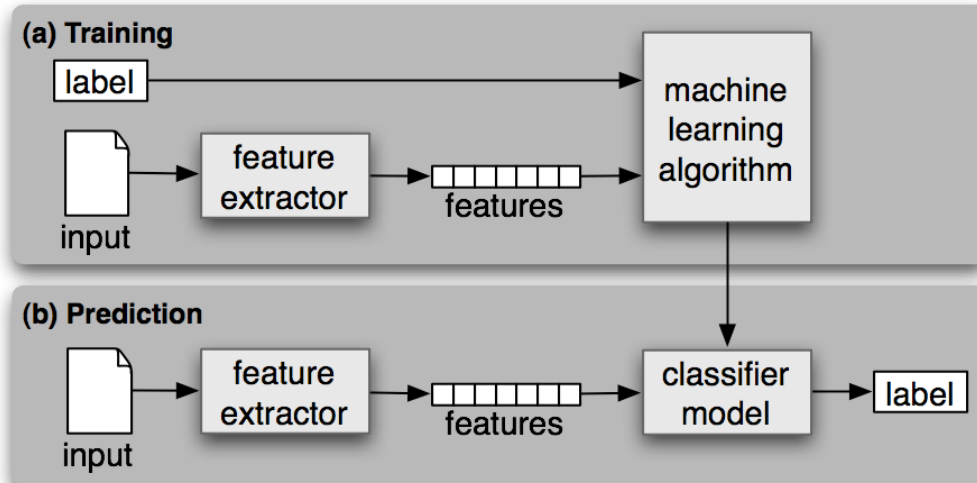
- **Feature extraction** is an important step for any machine learning problem.
- How do we **transform a given text into numerical form** so that it can be fed into NLP and ML algorithms?



<http://vku.udn.vn/>

3

VKU



<http://vku.udn.vn/>

4



Feature engineering

- **Feature engineering** steps **convert the raw data** into a format that can be **consumed by a machine**.
- **Two different approaches** taken in practice for feature engineering
 - (1) a classical NLP and traditional ML pipeline
 - (2) a DL pipeline

<http://vku.udn.vn/>

5



Classical NLP/ML Pipeline

- Task of **sentiment classification** on product reviews in e-commerce (positive or negative)
- **Count the number of positive and negative words** in each review

★★★★★ Cực kì hài lòng

Đã mua từ Tiki Trading Màu: Ánh Cực Quang

Tiki Trading giao hàng nhanh, đặt hôm trước hôm sau đã nhận được hàng. Sản phẩm giao được đóng gói cẩn thận (mặc dù dùng giấy tái chế nhưng sản phẩm không bị cấn móp). Chất lượng sản phẩm thì dùng lâu mới có nhận xét, nhưng dòng Note này khỏi phải bàn, great!

★★★★★ Trải nghiệm người dùng rất tốt

Đã mua từ Tiki Trading Màu: Ánh Cực Quang

Tôi nâng cấp từ S7 lên, máy đem lại trải nghiệm gần như hoàn hảo: chất lượng nghe gọi tốt, tiếng rất trong, loa ngoài hơi nhỏ. Giao diện người dùng trơn tru, thân thiện, không bị thừa thãi. Màn hình hiển thị tốt hơn hẳn. Mặt sau đẹp song cầm hơi trơn, bị bám vân tay. Máy có dán sẵn màn hình và kèm case như trong rất vừa. Hàng đặt trước nên được tặng tai nghe bud ố. Tuy nhiên, dùng tai nghe bud lâu hơi mỏi tai

<http://vku.udn.vn/>

6

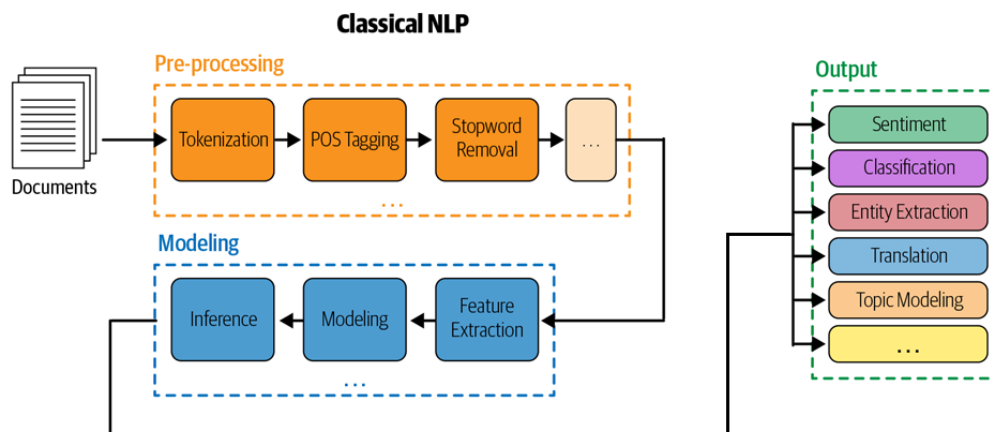
VKU Feature engineering

- Classical NLP/ML Pipeline
 - Usually **handcrafted** in the **classical ML pipeline**
 - **Statistical measures** for understanding if a feature is useful for a task or not
 - The features are **heavily inspired** by the task at hand as well as domain knowledge
 - The model remains **interpretable**—it's possible to **quantify** exactly how much **each feature is influencing** the model prediction
 - model performance and the model development cycle
 - noisy or unrelated feature can potentially harm the model's performance

<http://vku.udn.vn/>

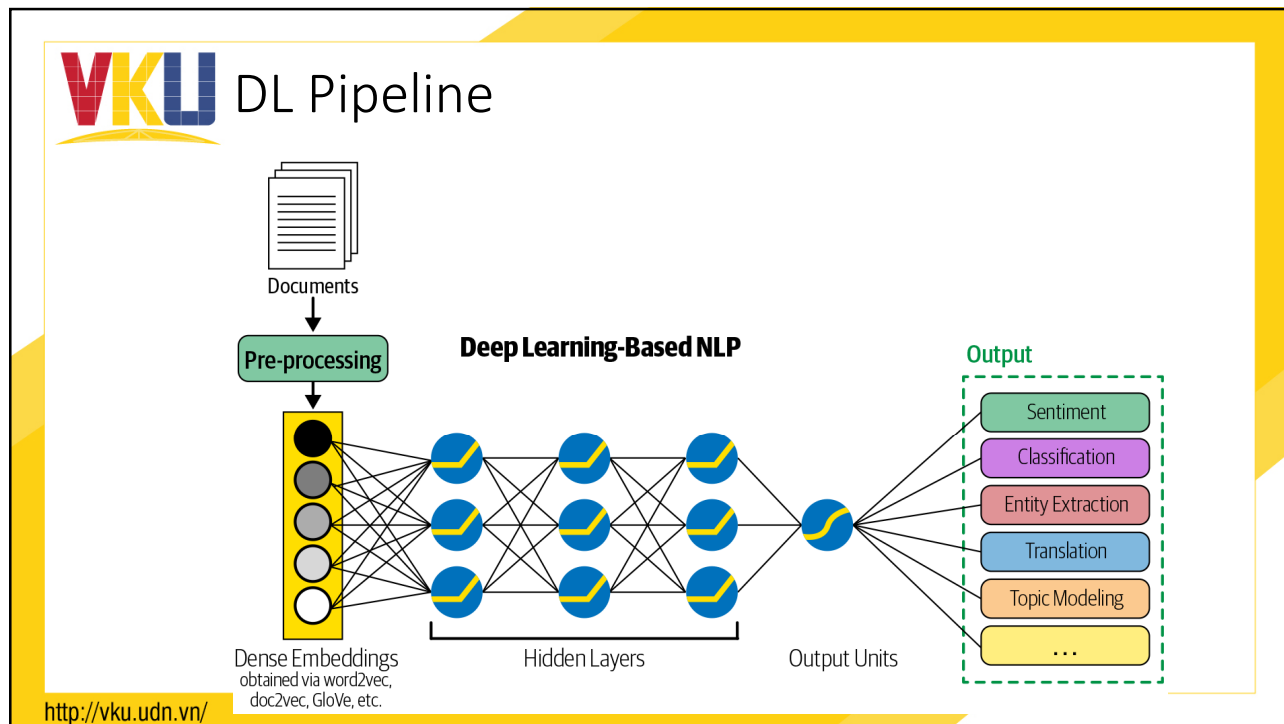
7

VKU • Classical NLP/ML Pipeline

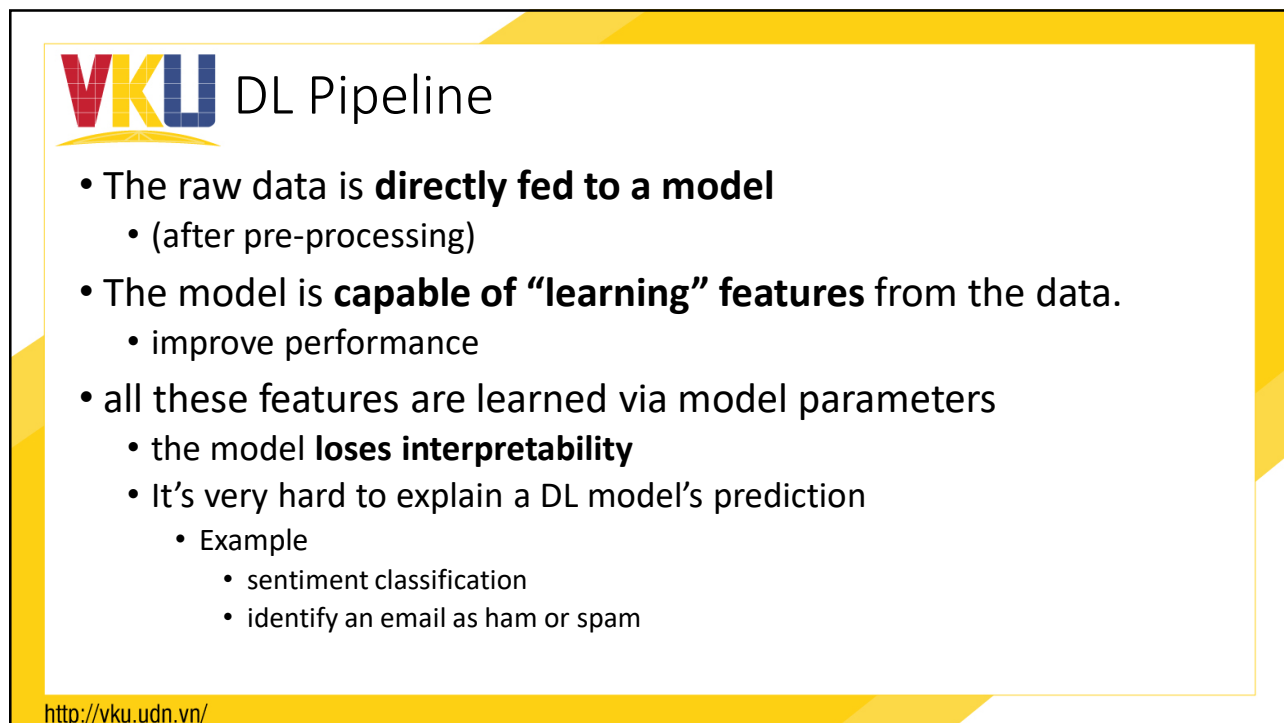


<http://vku.udn.vn/>

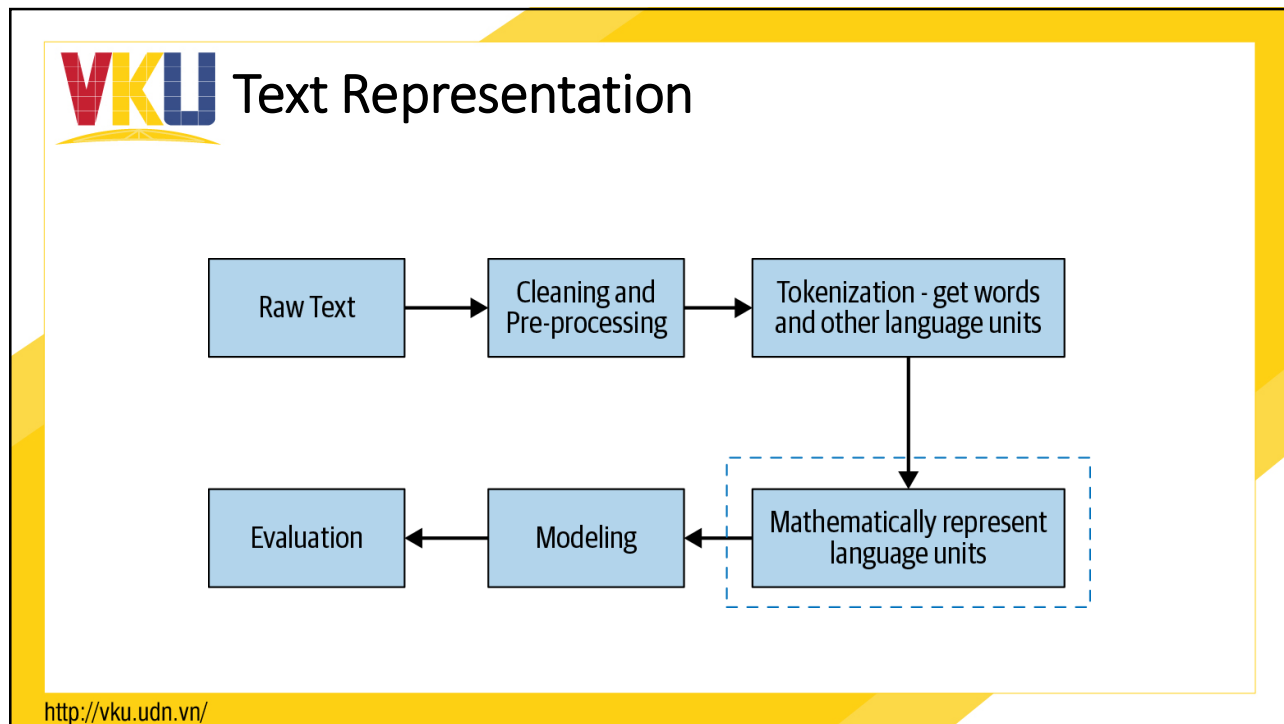
8



9



10



11

VKU

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Text Representation

Natural Language Processing

<http://vku.udn.vn/>

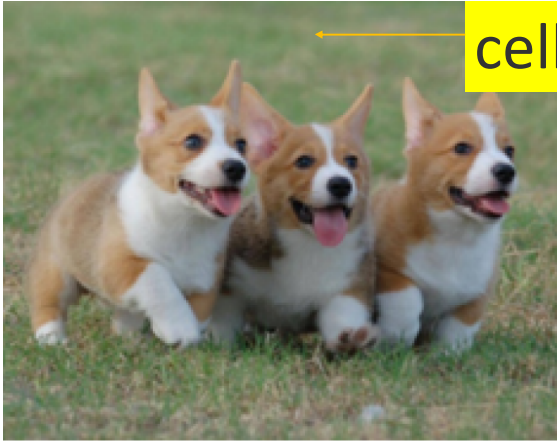
12



Numerical Representations of Data

- **Image:**

- Build a classifier that can distinguish images of cats from images of dogs



cell[i,j]

```

97 38 15 00 40 00 75 04 05 07 78 52 12 50 77 91 08
40 17 81 18 87 40 87 17 40 98 43 69 48 04 56 62 00
73 55 79 14 29 93 71 40 67 53 88 30 03 49 13 36 65
23 04 60 11 42 69 24 68 56 01 32 56 71 37 02 36 91
22 31 16 71 51 67 63 89 41 92 36 54 22 40 40 28 66 33 13 80
24 47 32 60 99 03 45 02 44 75 33 53 78 36 84 20 35 17 12 50
32 98 81 28 64 23 67 10 26 38 40 67 59 54 70 66 18 38 64 70
67 24 20 68 02 62 12 20 95 63 94 39 63 08 40 91 66 49 94 21
24 55 58 05 66 73 99 26 97 17 78 78 96 83 14 88 34 89 63 72
21 36 23 09 75 00 76 44 20 45 35 14 00 61 33 97 34 31 33 95
78 17 53 28 22 75 31 67 15 94 03 80 04 62 16 14 09 53 56 92
16 39 05 42 96 35 31 47 55 58 88 24 00 17 54 24 36 29 85 57
86 56 00 48 35 71 89 07 05 44 44 37 44 60 21 58 51 54 17 58
19 80 81 68 05 94 47 69 28 73 92 13 86 52 17 77 04 89 55 40
04 52 08 83 97 35 99 16 07 97 57 32 16 26 26 79 33 27 98 66
88 36 68 87 57 62 20 72 03 46 33 67 46 55 12 32 63 93 53 69
04 42 16 73 38 25 39 11 24 94 72 18 08 46 29 32 40 62 76 36
20 69 36 41 72 30 23 88 34 62 99 69 82 67 59 85 74 04 36 16
20 73 35 29 78 31 90 01 74 31 49 71 48 86 81 16 23 57 05 54
01 70 54 71 83 51 54 69 16 92 33 48 61 43 52 01 89 19 67 48

```

<http://vku.udn.vn/> What We See

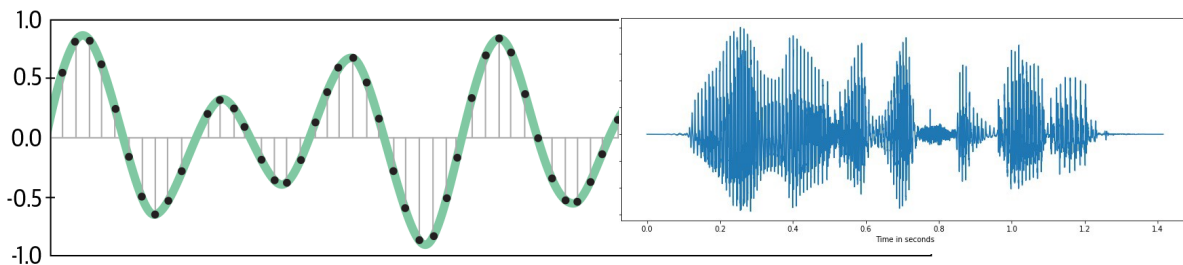
What Computers See

13



Numerical Representations of Data

- **Speech:** a numerical array representing the amplitude of a sound wave at fixed time intervals



```

[-1274, -1252, -1160, -986, -792, -692, -614, -429, -286, -134, -57, -41,
-169, -456, -450, -541, -761, -1067, -1231, -1047, -952, -645, -489, -448,
-397, -212, 193, 114, -17, -110, 128, 261, 198, 390, 461, 772, 948, 1451,
1974, 2624, 3793, 4968, 5939, 6057, 6581, 7302, 7640, 7223, 6119, 5461,
4820, 4353, 3611, 2740, 2004, 1349, 1178, 1085, 901, 301, -262, -499,
-488, -707, -1406, -1997, -2377, -2494, -2605, -2675, -2627, -2500, -2148,
-1648, -970, -364, 13, 260, 494, 788, 1011, 938, 717, 507, 323, 324, 325,
350, 103, -113, 64, 176, 93, -249, -461, -606, -909, -1159, -1307, -1544]

```

<http://vku.udn.vn/>

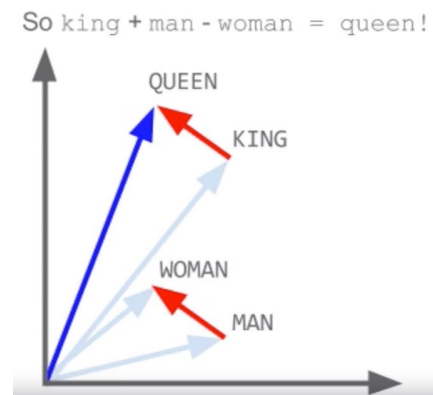
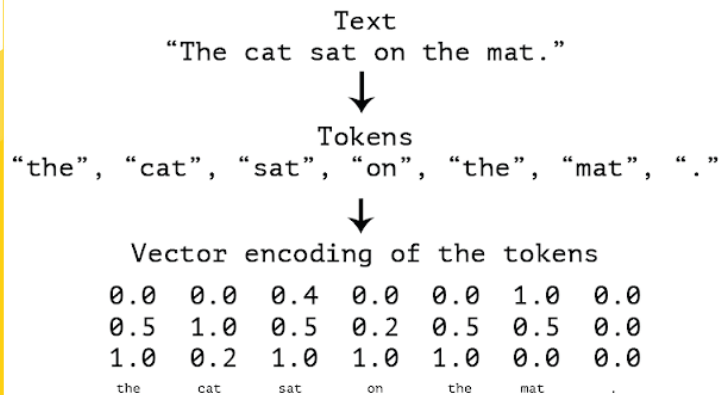
14



Text representation

- Text representation:

Given a piece of **text** → find a scheme to **represent it mathematically**



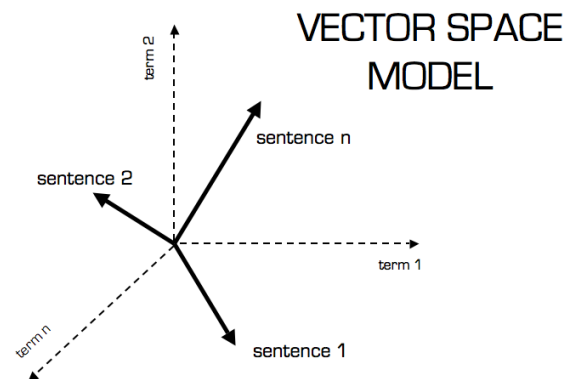
<http://vku.udn.vn/>

15



Vector Space Models - VSM

- VSM: Mô hình không gian vector**
- The text data must be **converted into some mathematical form**.
- represent text units** (characters, phonemes, words, phrases, sentences, paragraphs, and documents) **with vectors of numbers**.
- VSM: It's a **mathematical model** that **represents text units as vectors**



<http://vku.udn.vn/>

16

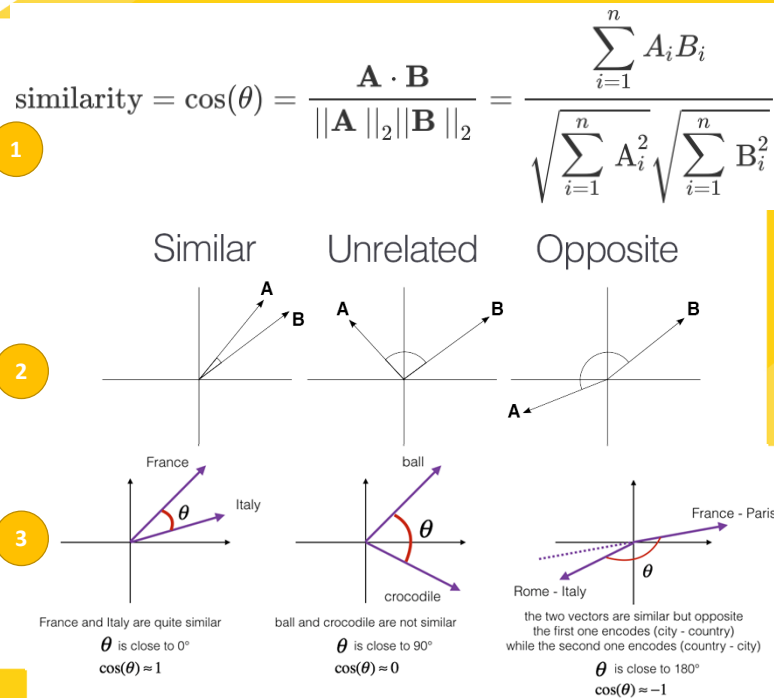
VKU VSM

- Calculation similarity between two text blobs is using **cosine similarity**
- $\cos(0^\circ) = 1$
- $\cos(180^\circ) = -1$
- the cosine monotonically decreasing from 0° to 180°

1

2

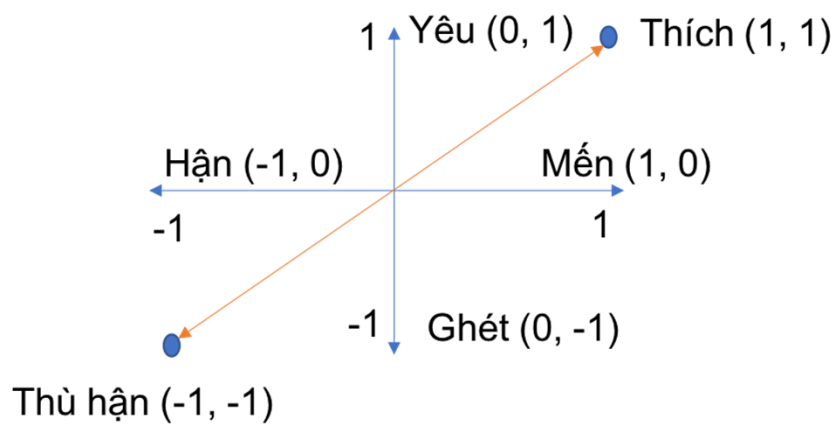
3



<http://vku.udn.vn/>

17

VKU



<http://vku.udn.vn/>

18



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Basic Vectorization Approaches

- + One-Hot Encoding
- + Bag of Words
- + Bag of N-Grams
- + TF-IDF

<http://vku.udn.vn/>

19



- text corpus
- document
- vocabulary (V)
- V-dimensional vector: $[0,0,0,0,0,0]$ / $[0,1,0,1,0,1]$ / $[1,1,1,1,1,1]$ (V=6)

- toy corpus:

four documents—D1 , D2 , D3 , D4

- vocabulary of this corpus

[dog, bites, man, eats, meat, food]

D1	Dog bites man.
D2	Man bites dog.
D3	Dog eats meat.
D4	Man eats food.

<http://vku.udn.vn/>

20



One-Hot Encoding

D1	Dog bites man.
D2	Man bites dog.
D3	Dog eats meat.
D4	Man eats food.

- **vocabulary** of this **corpus** is comprised of **six words**
 $V = [\text{dog, bites, man, eats, meat, food}]$
- **each word** w is given a **unique integer ID** w_{id} : value $1 \rightarrow |V|$
- **Each word** is then represented by a **V-dimensional binary vector** of 0s and 1s.
- Vector of w :
 - At index w_{id} : value=1
 - Others: value=0

<http://vku.udn.vn/>

21



One-Hot Encoding

$V = [\text{dog, bites, man, eats, meat, food}]$

- Map each of the six words to unique IDs:
 $\text{dog} = 1, \text{bites} = 2, \text{man} = 3, \text{meat} = 4, \text{food} = 5, \text{eats} = 6$
- **Dog** is represented as $[1\ 0\ 0\ 0\ 0\ 0]$
 $([\text{dog}, 0\text{-bites}, 0\text{-man}, 0\text{-eats}, 0\text{-meat}, 0\text{-food}])$
- **Bites** is represented as $[0\ 1\ 0\ 0\ 0\ 0]$
- **D1** is represented as
 $[[\underline{1\ 0\ 0\ 0\ 0\ 0}]\ [\underline{0\ 1\ 0\ 0\ 0\ 0}]\ [\underline{0\ 0\ 1\ 0\ 0\ 0}]]$

dog
bites
man

D1	Dog bites man.
D2	Man bites dog.
D3	Dog eats meat.
D4	Man eats food.

<http://vku.udn.vn/>

22



One-Hot Encoding

```
def get_onehot_vector(somestring):
    onehot_encoded = []
    for word in somestring.split():
        temp = [0]*len(vocab)
        if word in vocab:
            temp[vocab[word]-1] = 1
        onehot_encoded.append(temp)
    return onehot_encoded

get_onehot_vector(processed_docs[1])
```

Output: `[[0, 0, 1, 0, 0, 0], [0, 1, 0, 0, 0, 0], [1, 0, 0, 0, 0, 0]]`

<https://github.com/practical-nlp/practical-nlp-code/tree/master/Ch3>

<http://vku.udn.vn/>

23



One-Hot Encoding

- Pros and cons
 - **intuitive to understand** and **straightforward to implement**
 - The size of a one-hot vector is **directly proportional to size of the vocabulary**, and most real-world corpora have large vocabularies.
 - **not give a fixed-length** representation for text. For most learning algorithms, we need the feature vectors to be of the same length.
 - It treats **words as atomic units** and has no notion of (dis)similarity between words.
 - For example: run, ran, and apple.
 - **Out Of Vocabulary (OOV)** problem:
 - “man eats fruits”: The training data didn’t include “fruit” and there’s no way to represent it in our model

<http://vku.udn.vn/>

24



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Basic Vectorization Approaches

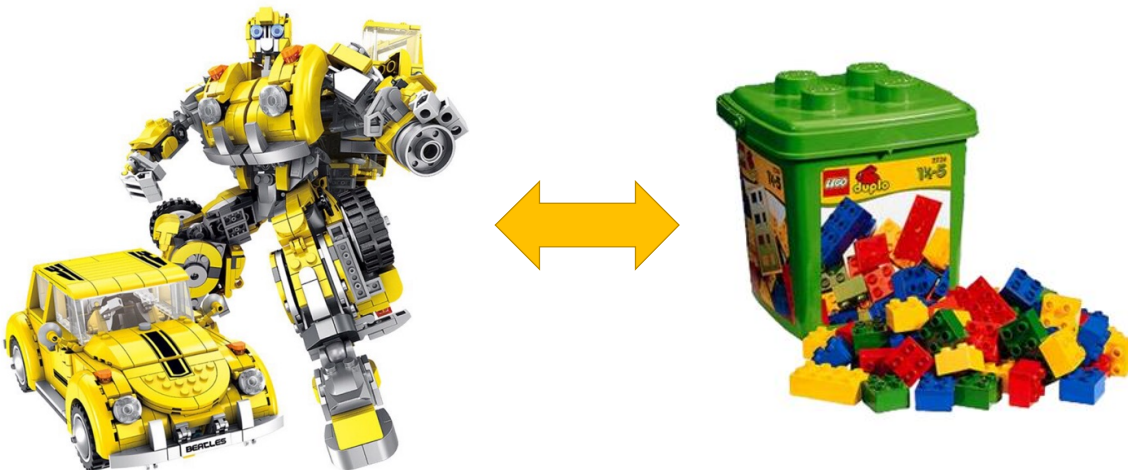
- + One-Hot Encoding
- + Bag of Words
- + Bag of N-Grams
- + TF-IDF

<http://vku.udn.vn/>

25



Bag of Words



<http://vku.udn.vn/>

26



Bag of Words - BoW

- Represent the text under consideration as a **bag (collection) of words** while **ignoring the order and context**
- assumes that the text belonging to a given class in the dataset is characterized by a unique set of words. If two text pieces have nearly the **same words**, then they belong to the **same bag** (class).
- BoW maps **words** to **unique integer IDs between 1 and $|V|$** .
- Each **document** → converted into a **vector of $|V|$ dimensions**
 where index i = **number of times the word w occurs** in the document
 → simply score each word in V by their occurrence count in the document

<http://vku.udn.vn/>

27



Bag of Words - BoW

- word IDs: dog = 1, bites = 2, man = 3, meat = 4, food = 5, eats = 6
- D1 becomes [1 1 1 0 0 0]
- D4 becomes [0 0 1 0 1 1]
- Dog and dog are friends.
- [0 2 0 0 0 0]

D1	Dog bites man.
D2	Man bites dog.
D3	Dog eats meat.
D4	Man eats food.

<http://vku.udn.vn/>

28

```

from sklearn.feature_extraction.text import CountVectorizer
count_vect = CountVectorizer()

#Build a BoW representation for the corpus
bow_rep = count_vect.fit_transform(processed_docs)

#Look at the vocabulary mapping
print("Our vocabulary: ", count_vect.vocabulary_)

#See the BoW rep for first 2 documents
print("BoW representation for 'dog bites man': ", bow_rep[0].toarray())
print("BoW representation for 'man bites dog: ", bow_rep[1].toarray())

#Get the representation using this vocabulary, for a new text
temp = count_vect.transform(["dog and dog are friends"])
print("Bow representation for 'dog and dog are friends':",
temp.toarray())

Our corpus: ['dog bites man', 'man bites dog', 'dog eats meat', 'man eats food']
Our vocabulary: {'dog': 1, 'bites': 0, 'man': 4, 'eats': 2, 'meat': 5, 'food': 3}
BoW representation for 'dog bites man': [[1 1 0 0 1 0]]
BoW representation for 'man bites dog: [[1 1 0 0 1 0]]
Bow representation for 'dog and dog are friends': [[0 2 0 0 0 0]]

```

<http://vku.udn.vn/>
<https://github.com/practical-nlp/practical-nlp-code/tree/master/Ch3>

29



Bag of Words - BoW

- Advantages of this encoding
 - BoW is fairly **simple to understand and implement**.
 - **documents having the same words** will have their vector representations **closer to each other**.
 - The vector space resulting from the BoW scheme captures the **semantic similarity of documents**. So if two documents have similar vocabulary, they'll be closer to each other in the vector space and vice versa.
 - We have a **fixed-length encoding** for any sentence of arbitrary length.

<http://vku.udn.vn/>

30



Bag of Words - BoW

- Disadvantages
 - The **size of the vector increases** with the size of the vocabulary → limiting the vocabulary to n number of the most frequent words
 - It does not capture the similarity between **different words** that mean the **same thing**.
 - “I run”, “I ran”, and “I ate”.
 - BoW vectors of all three documents will be equally apart.
 - Problem of Out Of Vocabulary words
 - “bag” of words — **word order** information is lost in this representation. (D1 & D2)

D1

Dog bites man.

D2

Man bites dog.

<http://vku.udn.vn/>

31



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Basic Vectorization Approaches

- + One-Hot Encoding
- + Bag of Words
- + Bag of N-Grams
- + TF-IDF

<http://vku.udn.vn/>

32



- Sao, bảo, nó, không, đến
- Sao không bảo nó đến?
- Bảo nó sao không đến?
- Đến sao không bảo nó?
- Không bảo sao nó đến?
- Nó bảo sao không đến?
- ...

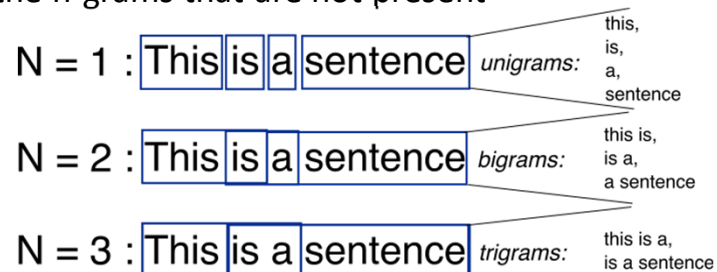
<http://vku.udn.vn/>

33



Bag of N-Grams - BoN

- Breaking text into chunks of **n contiguous words** (or tokens)
- Each **chunk** is called an **n-gram**
- Each **document** → represented by a **vector of length $|V|$** .
- This vector contains the **frequency counts of ngrams** and zero for the n-grams that are not present



<http://vku.udn.vn/>

34



Bigram: 2-gram

- {dog bites, bites man, man bites, bites dog, dog eats, eats meat, man eats, eats food}

D1	Dog bites man.
D2	Man bites dog.
D3	Dog eats meat.
D4	Man eats food.

- BoN representation: **eight-dimensional vector** for each document
- D1 : [1,1,0,0,0,0,0,0], D2 : [0,0,1,1,0,0,0,0]
- **BoW** scheme is a special case of the **BoN** scheme, **with n=1**
- n=1: unigram
- n=2: "bigram model"
- n=3: "trigram model"

<http://vku.udn.vn/>

35



```
#n-gram vectorization example with count vectorizer and uni, bi,
trigrams
count_vect = CountVectorizer(ngram_range=(1,3))

#Build a BOW representation for the corpus
bow_rep = count_vect.fit_transform(processed_docs)

#Look at the vocabulary mapping
print("Our vocabulary: ", count_vect.vocabulary_)

#Get the representation using this vocabulary, for a new text
temp = count_vect.transform(["dog and dog are friends"])
print("Bow representation for 'dog and dog are friends':",
temp.toarray())
```

<http://vku.udn.vn/>

36



Bag of N-Grams - BoN

- pros and cons of BoN
 - It captures some **context and word-order** information in the form of n-grams
 - resulting vector space is able to capture some **semantic similarity**
 - As n increases, dimensionality (and therefore sparsity) only increases rapidly
 - **OOV problem**

<http://vku.udn.vn/>

37



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Basic Vectorization Approaches

- + One-Hot Encoding
- + Bag of Words
- + Bag of N-Grams
- + TF-IDF

<http://vku.udn.vn/>

38

VKU TF-IDF

- One Hot, BoW, BoN: all the words in the text are treated as **equally important**—there's no notion of **some words** in the document being more **important than others**
- TF-IDF: term frequency–inverse document frequency
- quantify the **importance of a given word relative to other words** in the document and in the corpus

<http://vku.udn.vn/>

39

VKU TF-IDF

- if a **word w**
 - **appears many times** in a document **d_i**
 - but does **not occur much** in the rest of the documents **d_j** in the corpus
- then the **word w**
 - must be of **great importance** to the **document d_i**
- **TF**
- **IDF**
- **→ TF-IDF score = $TF * IDF$**

<http://vku.udn.vn/>

40



TF (term frequency)

- TF: measures **how often a term** or word occurs in **a given document**
- TF of a term **t** in a document **d** is defined as:

$$\text{TF}(t, d) = \frac{(\text{Number of occurrences of term } t \text{ in document } d)}{(\text{Total number of terms in the document } d)}$$

<http://vku.udn.vn/>

41



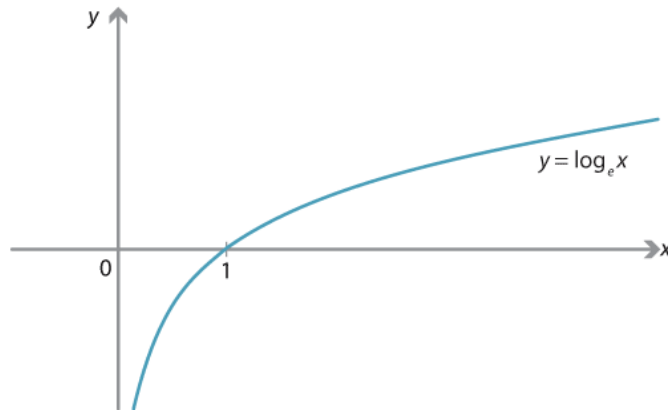
IDF (inverse document frequency)

- IDF: measures the importance of the term across a corpus
 - **weighs down** the terms that are **very common** across a corpus
 - **weighs up** the **rare terms**
- IDF of a term **t** is calculated as follows:

$$\text{IDF}(t) = \log_e \frac{(\text{Total number of documents in the corpus})}{(\text{Number of documents with term } t \text{ in them})}$$

<http://vku.udn.vn/>

42



<http://vku.udn.vn/>

43



TF-IDF

$$\text{TF}(t, d) = \frac{(\text{Number of occurrences of term } t \text{ in document } d)}{(\text{Total number of terms in the document } d)}$$

$$\text{IDF}(t) = \log_e \frac{(\text{Total number of documents in the corpus})}{(\text{Number of documents with term } t \text{ in them})}$$

$$\text{TF-IDF score} = \text{TF} * \text{IDF}$$

<http://vku.udn.vn/>

44



Let's compute TF-IDF scores for our toy corpus

- The size of our corpus is $N=4$
- Vocabulary:
- $V: [\text{dog, bites, man, eats, meat, food}]$
- $|V| = 6$
- D1
 - Dog:
 - $TF = 1/3 = 0,33$
 - $IDF = \log_2(4/3) = 0,41$
 - $TF-IDF = TF * IDF = 0,33 * 0,41 = 0,136$

D1	Dog bites man.
D2	Man bites dog.
D3	Dog eats meat.
D4	Man eats food.

<http://vku.udn.vn/>

45



Example

$$tf("this", d_1) = \frac{1}{5} = 0.2$$

$$tf("this", d_2) = \frac{1}{7} \approx 0.14$$

$$idf("this", D) = \log\left(\frac{2}{2}\right) = 0$$

$$tfidf("this", d_1, D) = 0.2 \times 0 = 0$$

$$tfidf("this", d_2, D) = 0.14 \times 0 = 0$$

Document 1		Document 2	
Term	Term Count	Term	Term Count
this	1	this	1
is	1	is	1
a	2	another	2
sample	1	example	3

<http://vku.udn.vn/>

46



$$\text{tf}(\text{"example"}, d_1) = \frac{0}{5} = 0$$

$$\text{tf}(\text{"example"}, d_2) = \frac{3}{7} \approx 0.429$$

$$\text{idf}(\text{"example"}, D) = \log\left(\frac{2}{1}\right) = 0.301$$

$$\text{tfidf}(\text{"example"}, d_1, D) = 0 \times 0.301 = 0$$

$$\text{tfidf}(\text{"example"}, d_2, D) = 0.429 \times 0.301 \approx 0.129$$

Document 1	
Term	Term Count
this	1
is	1
a	2
sample	1

Document 2	
Term	Term Count
this	1
is	1
another	2
example	3

<http://vku.udn.vn/>

47



```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer()
bow_rep_tfidf = tfidf.fit_transform(processed_docs)
print(tfidf.idf_) #IDF for all words in the vocabulary
print(tfidf.get_feature_names()) #All words in the vocabulary.

temp = tfidf.transform(["dog and man are friends"])
print("Tfidf representation for 'dog and man are friends':\n",
      temp.toarray())
```

<http://vku.udn.vn/>

48



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Distributed Representations

+ Word Embeddings

<http://vku.udn.vn/>

49



Distributed Representations

- **Distributional similarity**

- **the meaning of a word** can be understood from the **context** in which the word appears
- connotation: meaning is defined by context

- **Distributional hypothesis**

- words that occur in **similar contexts** have **similar meanings**
- “dog” and “cat” occur in similar contexts - there must be a strong similarity between the meanings of these two words.
- VSM: if **two words often occur in similar context**, then their corresponding representation **vectors** must also be **close to each other**

<http://vku.udn.vn/>

50



-Distributed Representations

- Distributional representation
 - representation schemes that are obtained based on distribution of words from the context in which the words appear
 - These schemes are based on distributional hypotheses
 - The distributional property is induced from context
 - distributional representation schemes use highdimensional vectors to represent words
 - The dimension of this matrix is equal to the size of the vocabulary of the corpus
 - one-hot, bag of words, bag of n-grams, and TF-IDF

<http://vku.udn.vn/>

51



Distributed Representations

- **Distributed representation**
 - based on the distributional hypothesis
 - distributed representation schemes significantly **compress** the **dimensionality**.
 - This results in **vectors** that are **compact** (i.e., low dimensional) and **dense** (i.e., hardly any zeros)

<http://vku.udn.vn/>

52



Distributed Representations

• Embedding

- For the set of words in a corpus, embedding is a mapping between vector space coming from distributional representation to vector space coming from distributed representation

• Vector semantics

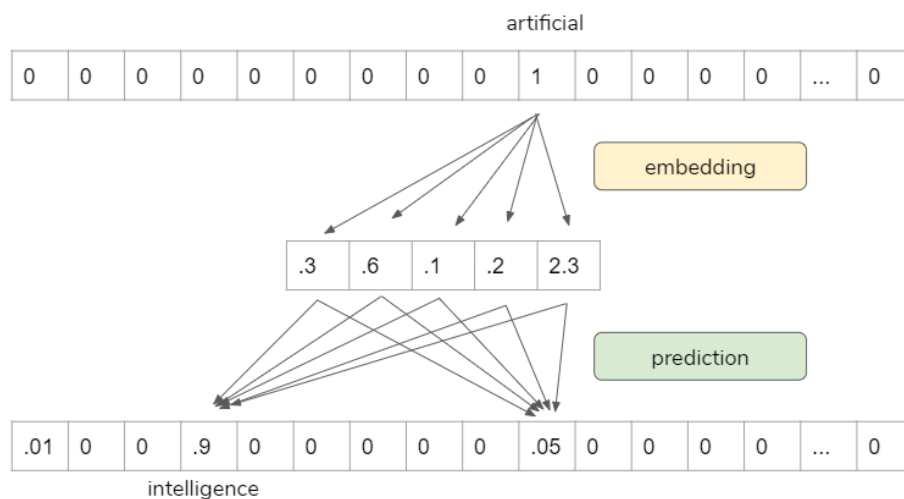
- This refers to the set of NLP methods that aim to learn the word representations based on distributional properties of words in a large corpus

<http://vku.udn.vn/>

53



Embedding



<http://vku.udn.vn/>

54



Word Embeddings

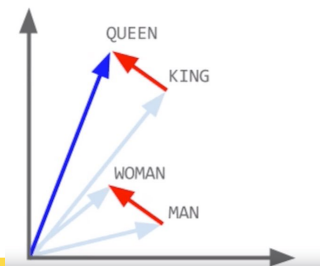
• Distributional similarities between words

- given the word “USA,” distributionally similar words could be other countries (e.g., Canada, Germany, India, etc.) or cities in the USA
- given the word “beautiful,” words that share some relationship with this word (e.g., synonyms, antonyms) could be considered distributionally similar words.

• These are words that are likely to **occur in similar contexts**

- Word representation model known as “Word2vec,” based on “distributional similarity”

$$\text{King} - \text{Man} + \text{Woman} \approx \text{Queen}$$



<http://vku.udn.vn/>

55



Word2vec

```
>>> print(new_model['dep'])
[ 0.5671392  -0.10314388 -0.09728424 -0.1489985   1.0291566  -0.00815625
 -0.8848648   0.6005959   1.3104095  -0.94979966 -0.46398002  0.63081545
  1.2366766  -1.1794204   0.60577285  0.18720543 -1.8238723   0.04002761
 -2.5429482   0.08705863  0.75452757  1.401774   -0.78432107  1.0807766
 -0.5465884   0.9397089  -1.2304566   1.0299896  -2.9703097   0.97884893
  2.3262892  -2.398893   0.86914957 -2.9250796  -0.5910089  -2.960267
 -0.8937288   4.084918   1.3665831  -2.3544352   1.1426593  -0.14586192
  2.8293912  -1.3207251  -0.99239624  0.00454006  2.1511614   2.4628458
 -1.2247262  -0.60409373  2.8358223  -0.17071097  2.3190877  -0.4726599
 -0.7420152   0.34608263 -2.8055074   2.3669877  -0.8600275  -1.5051168
  0.14877409  2.3596442  -0.18270311 -0.92651314  1.1652614  -2.4582791
 -0.60305524  2.140667   0.70670056  1.5177376   0.7195108   0.9756385
  0.45244128  1.8552047   0.7261587  -2.4192975  -0.76993984  1.6667477
  1.3484783  -1.0264692  -2.1513073  -1.3487064  -2.1187088  -1.3888106
 -1.2047982  -0.1363161  -0.10072072 -0.7533901  -1.3453391  -1.2854313
  0.55728    -0.9712059  -1.0520716  -1.1136116  -2.0543003   0.25404793
  1.9128581  -2.0700736  -0.7122985  -0.2631153 ]
```

<http://vku.udn.vn/>

56

VKU Word2vec

- Word2vec:
 - semantical rich relationships
 - the learned word representations are low dimensional ($|V| \sim 50-500$)
 - dense (most values in these vectors are non-zero)
- These representations are also called “embeddings.”
- Word2Vec
 - CBOW
 - Skip-gram

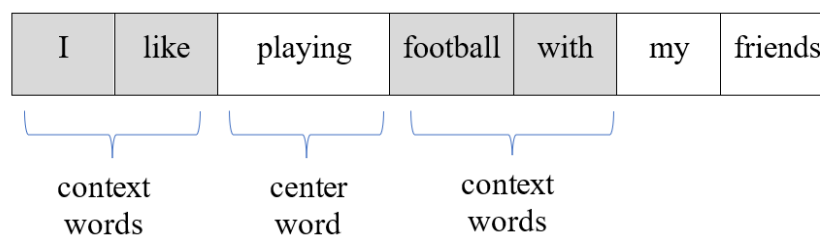
<http://vku.udn.vn/>

57

VKU CBOW

- CBOW: **predict** the “**center**” word from the words in its **context**.

The quick brown fox jumped over the lazy dog



<http://vku.udn.vn/>

58

VKU CBOW

- run a **sliding window** of size $2k+1$ over the text corpus ($k=2$)
- Each position: marks the set of $2k+1$ words
 - The **center word** in the window is the **target**,
 - **k words** on either side of the center word form the context
- **one data point (X, Y)**
 - context is the X
 - the target word is the Y
- **shift the window** to the right on the corpus by **one word** and repeat the process → **get the next data point**

<http://vku.udn.vn/>

59



- Window size: $2k+1 = 5$

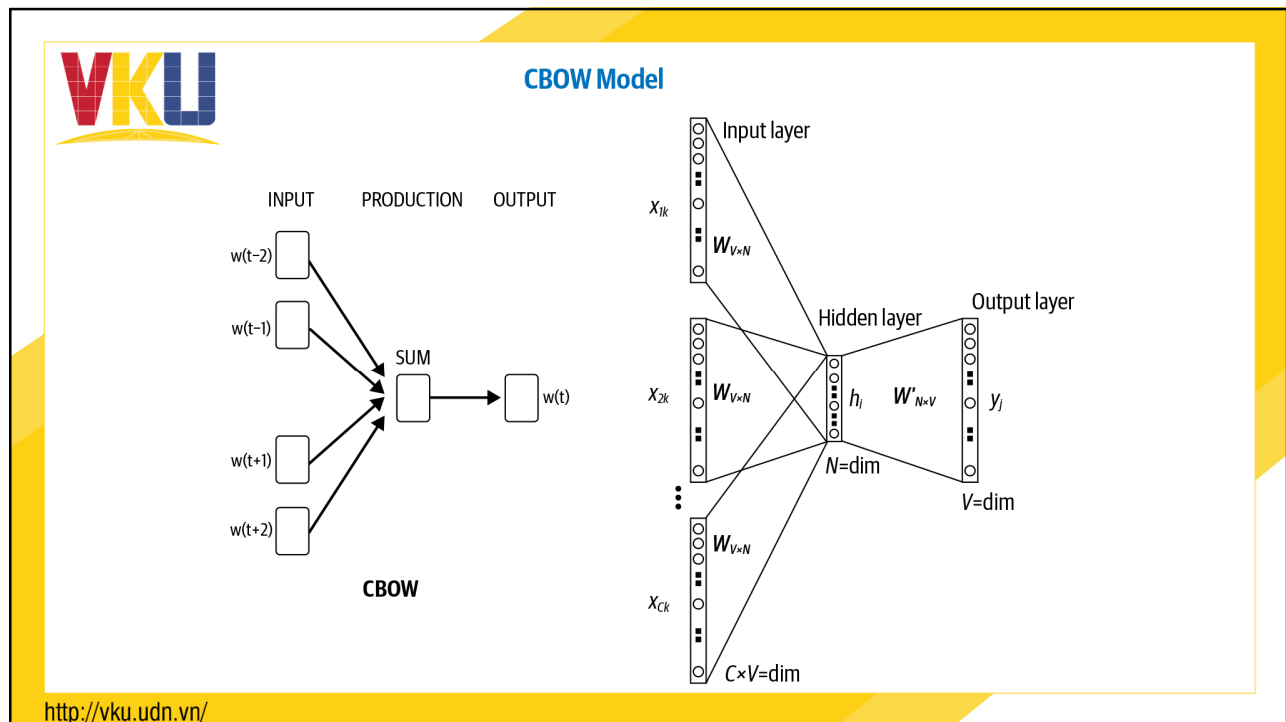
Source Text

Training Samples (context, target)

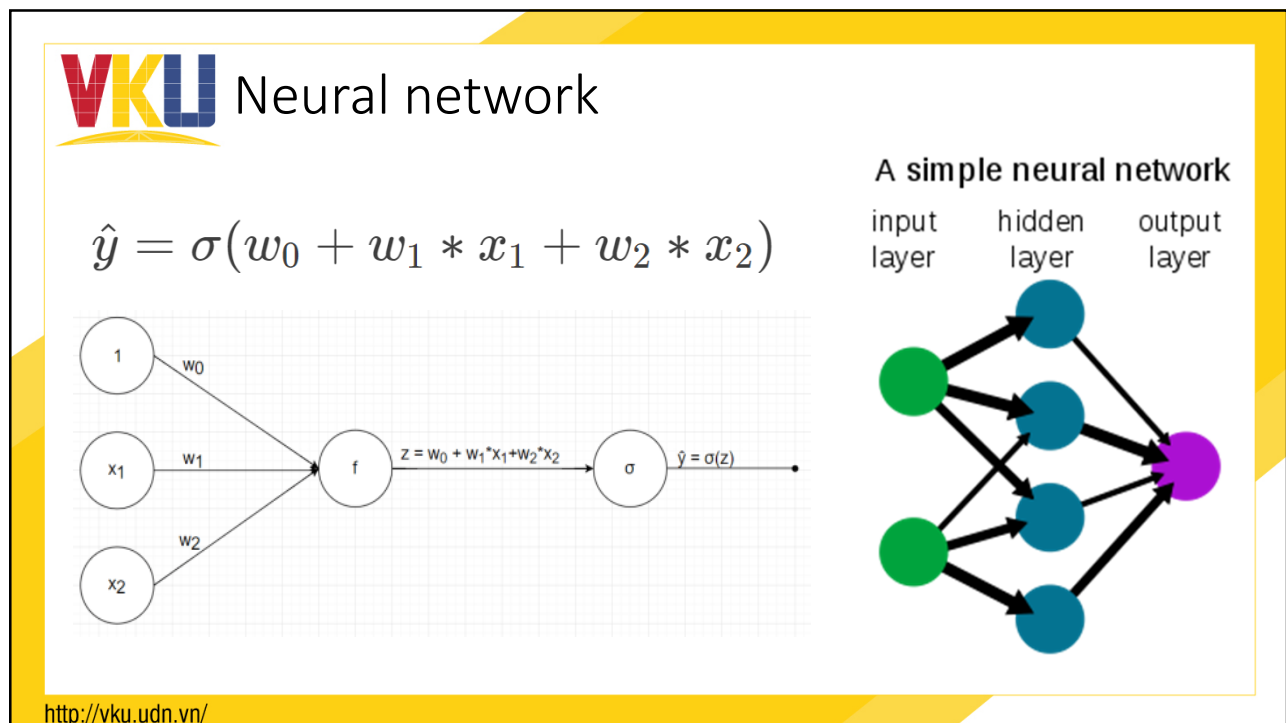
The quick brown fox jumps over the lazy dog.	→	((quick, brown), The)
The quick brown fox jumps over the lazy dog.	→	((The, brown, fox), quick)
The quick brown fox jumps over the lazy dog.	→	((The, quick, fox, jumps), brown)
The quick brown fox jumps over the lazy dog.	→	((quick, brown, jumps, over), fox)

<http://vku.udn.vn/>

60



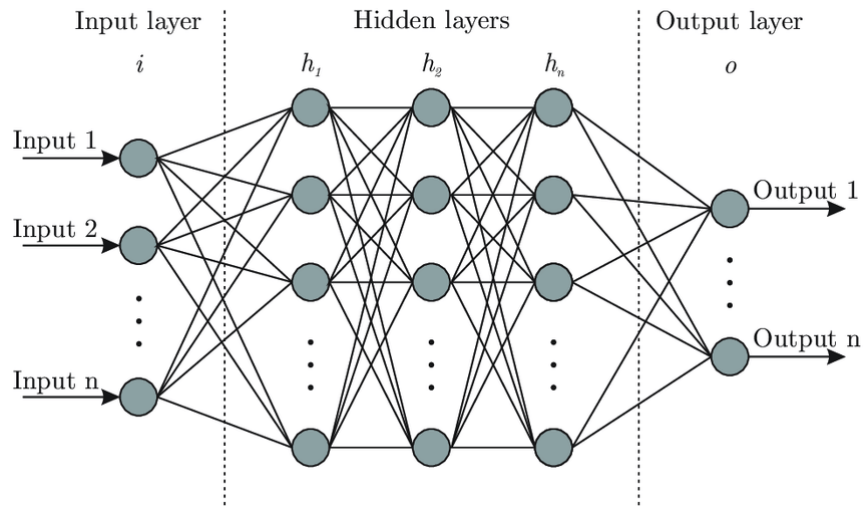
61



62



Neural network



<http://vku.udn.vn/>

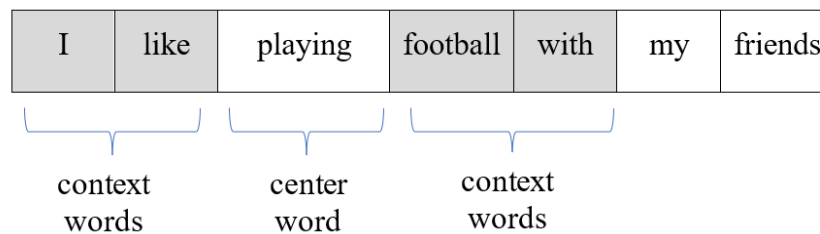
63



CBOW

- CBOW: **predict** the “**center**” word from the words in its **context**.

The quick brown fox jumped over the lazy dog

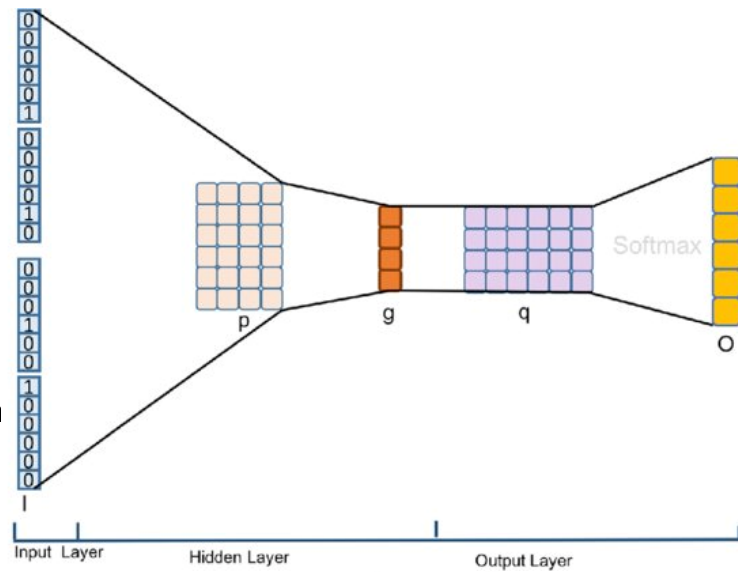


<http://vku.udn.vn/>

64

VKU CBOW

- word embedding dimension $n = 4$
- vocabulary size $|V| = 6$
- window size 5 ($k = 2$).
- input layer: $\text{Word}_i \in |V| \times 1$ is a one-hot vector
- hidden layer: $p \in \mathbb{R}^{|V| \times n}$ and $q \in n \times |V|$, g
- output layer: vector of probabilities $O \in |V| \times 1$ specifies the likelihood of each word to be the target word in the context window

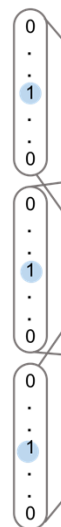


<http://vku.udn.vn/>

65

VKU CBOW

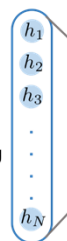
Input



N

Matrix W

Hidden

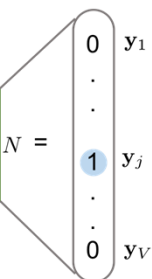


N -dimension vector
(Average of vectors of
all input words)

V

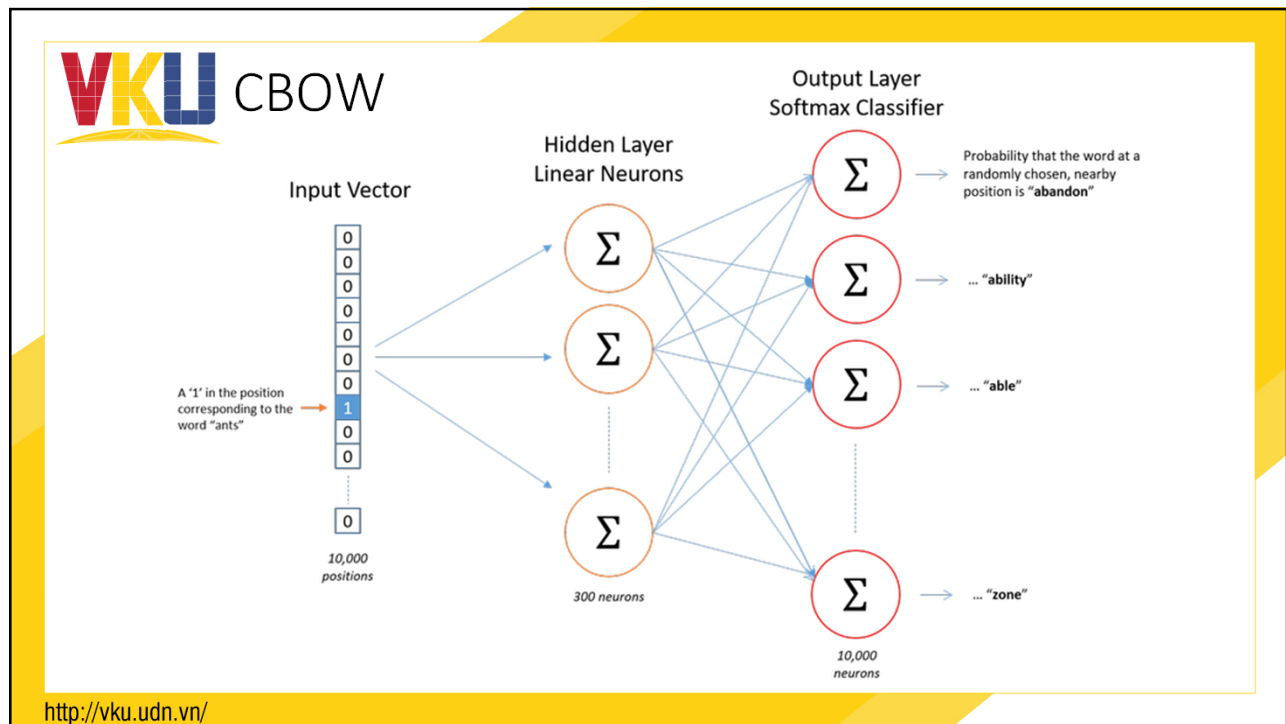
Matrix W'

Output
softmax

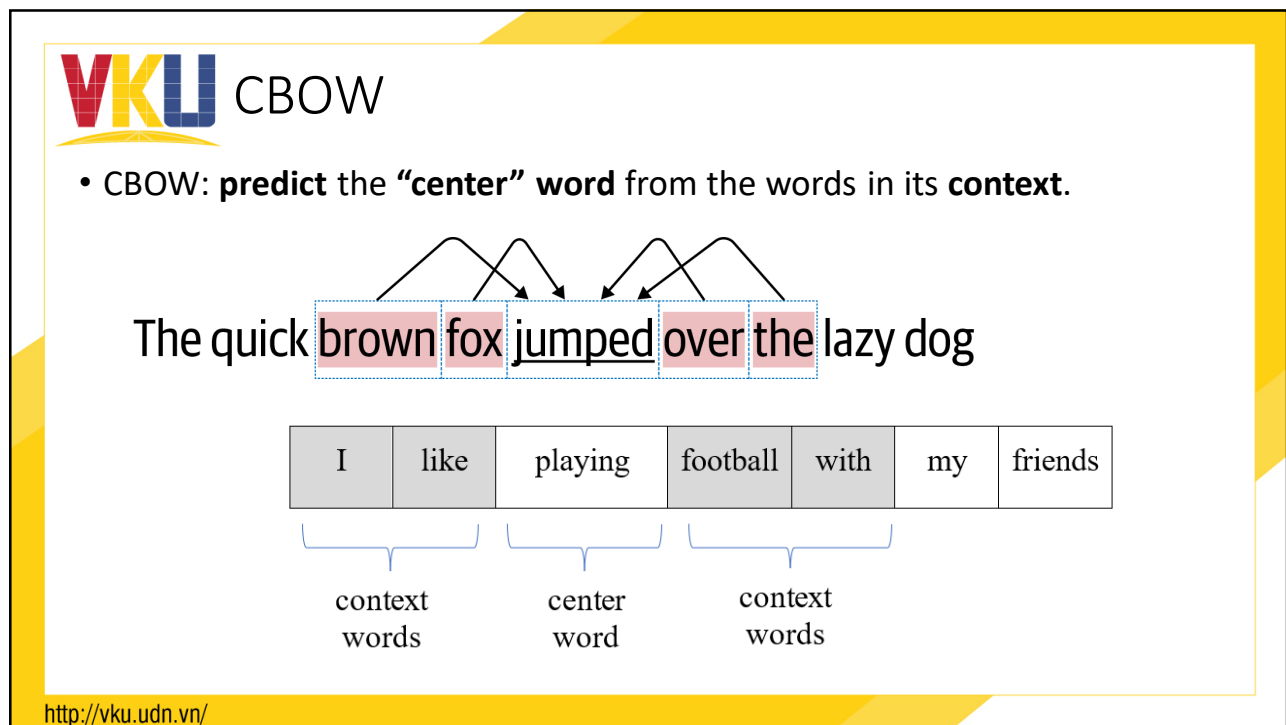


<http://vku.udn.vn/>

66



67

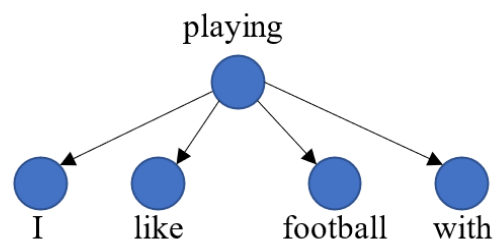
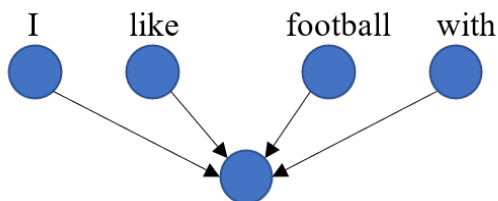


68

VKU SkipGram

- predict the context words from the center word

The quick brown fox jumped over the lazy dog



<http://vku.udn.vn/>

69

VKU SkipGram: Prepare data set

The quick brown fox jumped over the lazy dog

- run a sliding window of size $2k+1$ over the text corpus
→ set of $2k+1$ words
- **center word** in the window is the **X**,
- **k words** on either side of the center word are **Y**
→ $2k$ data points
- shift the window to the right on the corpus by one word
 - repeat the process

<http://vku.udn.vn/>

70



- Window size = 5

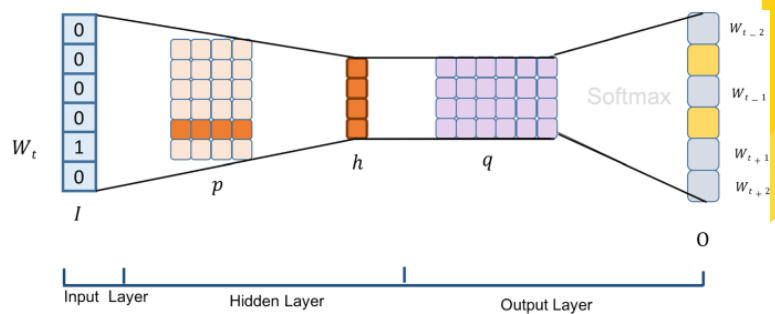
Source Text	Training Samples (target, context)
The quick brown fox jumps over the lazy dog.	(the, quick) (the, brown)
The quick brown fox jumps over the lazy dog.	(quick, the) (quick, brown) (quick, fox)
The quick brown fox jumps over the lazy dog.	(brown, the) (brown, quick) (brown, fox) (brown, jumps)
The quick brown fox jumps over the lazy dog.	(fox, quick) (fox, brown) (fox, jumps) (fox, over)

<http://vku.udn.vn/>

71



- word embedding dimension $n = 4$,
- vocabulary size $|V| = 6$
- window size 5 ($k = 2$).
- a vector of probabilities $O \in |V| \times 1$ in the output layer specifies the likelihood of each word to appear in the context window



<http://vku.udn.vn/>

72



```
#Import a test data set provided in gensim to train a model
from gensim.test.utils import common_texts
#Build the model, by selecting the parameters.
our_model = Word2Vec(common_texts, size=10, window=5, min_count=1,
workers=4)
#Save the model
our_model.save("tempmodel.w2v")
#Inspect the model by looking for the most similar words for a test word.
print(our_model.wv.most_similar('computer', topn=5))
#Let us see what the 10-dimensional vector for 'computer' looks like.
print(our_model['computer'])
```

<http://vku.udn.vn/>

73



PRE-TRAINED WORD EMBEDDINGS

- Some of the most popular pre-trained embeddings:
 - Word2vec by Google
 - GloVe by Stanford
 - Fasttext embeddings by Facebook
- available for various dimensions like $d = 25, 50, 100, 200, 300, 600$

<http://vku.udn.vn/>

74



```
from gensim.models import Word2Vec, KeyedVectors
pretrainedpath = "NLPBookTut/GoogleNews-vectors-negative300.bin"
w2v_model = KeyedVectors.load_word2vec_format(pretrainedpath,
binary=True)
print('done loading Word2Vec')
print(len(w2v_model.vocab)) #Number of words in the vocabulary.
print(w2v_model.most_similar('beautiful'))
w2v_model['beautiful']
```

- most_similar('beautiful')

```
[('gorgeous', 0.8353004455566406),
 ('lovely', 0.810693621635437),
 ('stunningly_beautiful', 0.7329413890838623),
 ('breathtakingly_beautiful', 0.7231341004371643),
 ('wonderful', 0.6854087114334106),
 ('fabulous', 0.6700063943862915),
 ('loveliest', 0.6612576246261597),
 ('prettiest', 0.6595001816749573),
 ('beatiful', 0.6593326330184937),
 ('magnificent', 0.6591402292251587)]
```

<http://vku.udn.vn/>

75



- the last line returns the
- embedding vector of the word “beautiful”

```
[6] #What is the vector representation for a word?
w2v_model['beautiful']
```

```
array([-0.01831055,  0.05566406, -0.01153564,  0.07275391,  0.15136719,
        -0.06176758,  0.20605469, -0.15332031, -0.05908203,  0.22851562,
        -0.06445312, -0.22851562, -0.09472656, -0.03344727,  0.24707031,
         0.05541992, -0.00921631,  0.1328125 , -0.15429688,  0.08105469,
        -0.07373047,  0.24316406,  0.12353516, -0.09277344,  0.08203125,
         0.06494141,  0.15722656,  0.11279297, -0.0612793 , -0.296875 ,
        -0.13378906,  0.234375 ,  0.09765625,  0.17773438,  0.06689453,
        -0.27539062,  0.06445312, -0.13867188, -0.08886719,  0.171875 ,
         0.07861328, -0.10058594,  0.23925781,  0.03808594,  0.18652344,
        -0.11279297,  0.22558594,  0.10986328, -0.11865234,  0.02026367,
         0.11376953,  0.09570312,  0.29492188,  0.08251953, -0.05444336,
        -0.0090332 , -0.0625 , -0.17578125, -0.08154297,  0.01062012,
        -0.04736328, -0.08544922, -0.19042969, -0.30273438,  0.07617188,])
```

<http://vku.udn.vn/>

76



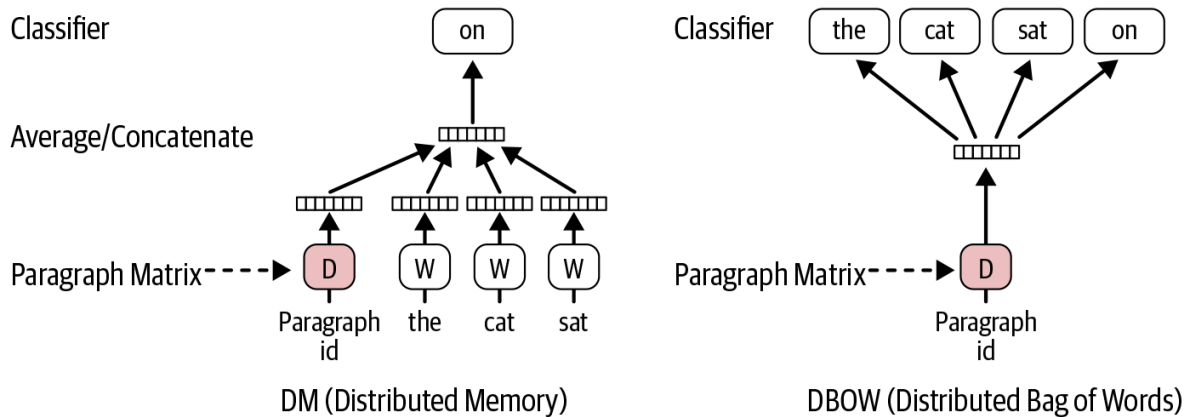
- Word Embeddings
 - Word2Vec
 - FastText
 - GloVe
- Distributed Representations Beyond Words and Characters
 - Doc2vec
 - Doc2vec is based on the paragraph vectors framework [21] and is implemented in genism
- Universal Text Representations
 - ELMo
 - BERT

<http://vku.udn.vn/>

77



Doc2vec



<http://vku.udn.vn/>

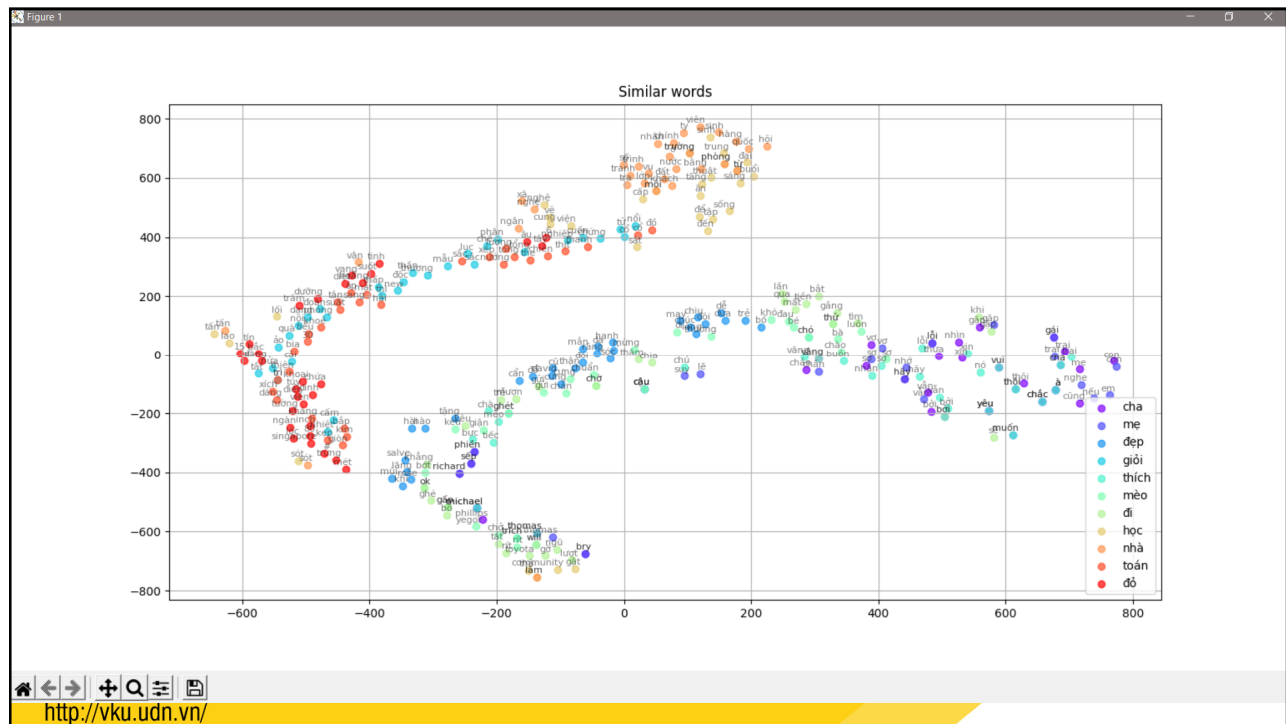
78



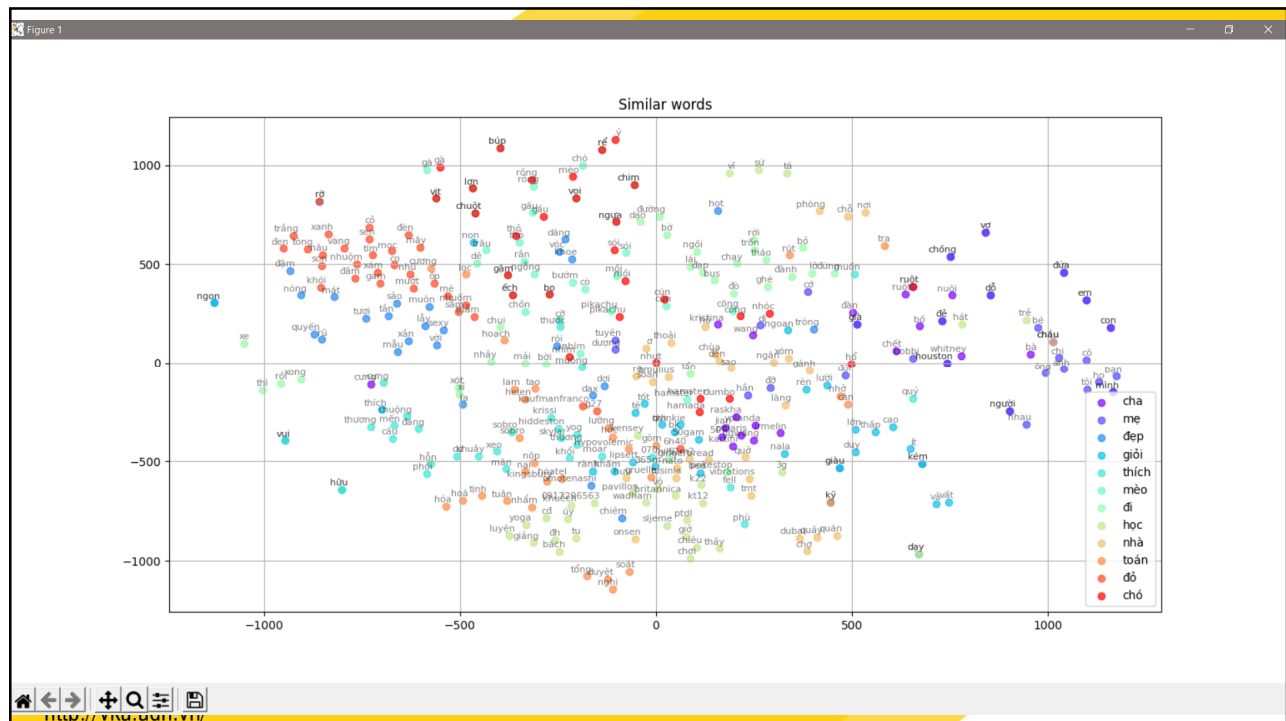
- Similar words



40



81

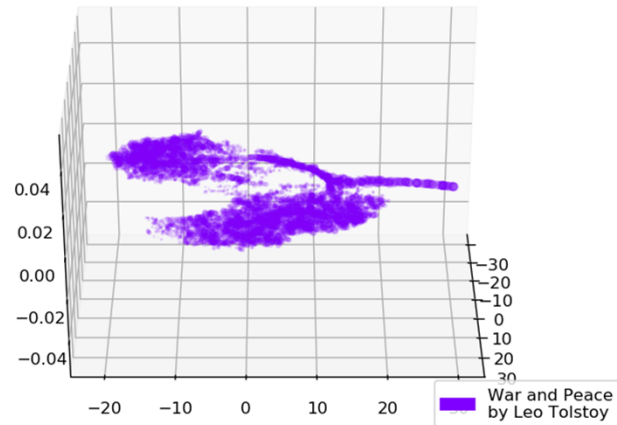


82



Visualizing Embeddings

Visualizing Word Embeddings using t-SNE



<http://vku.udn.vn/>

83



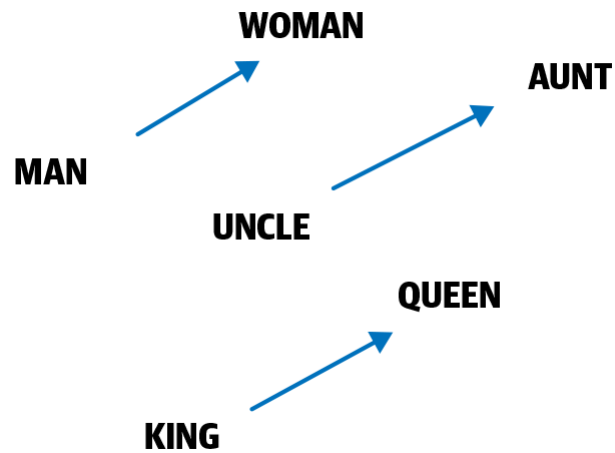
- <https://projector.tensorflow.org/>

<http://vku.udn.vn/>

84



Visualizing Embeddings

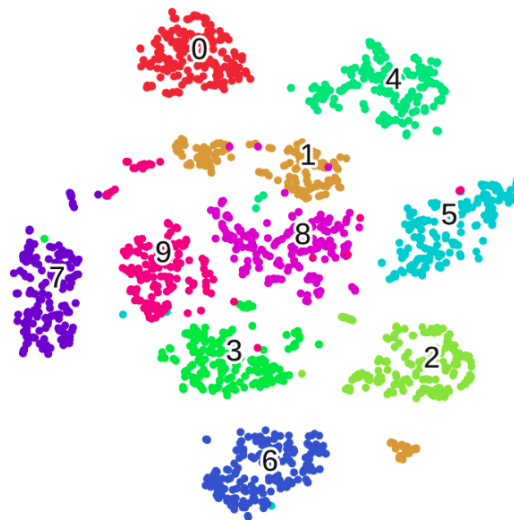


<http://vku.udn.vn/>

85



Visualizing MNIST data using t-SNE

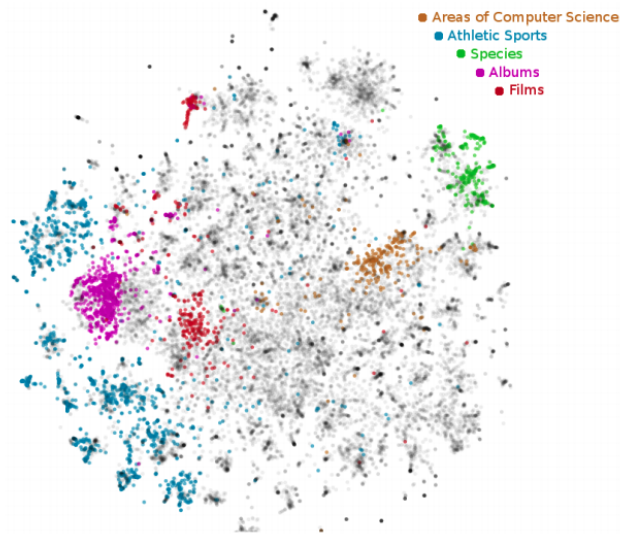


<http://vku.udn.vn/>

86



Visualization of Wikipedia document vectors

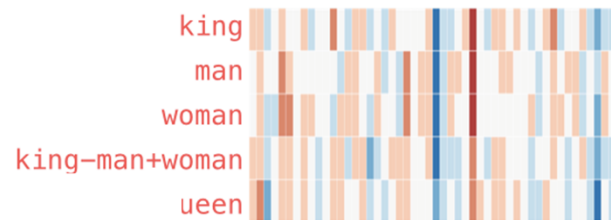


<http://vku.udn.vn/>

87

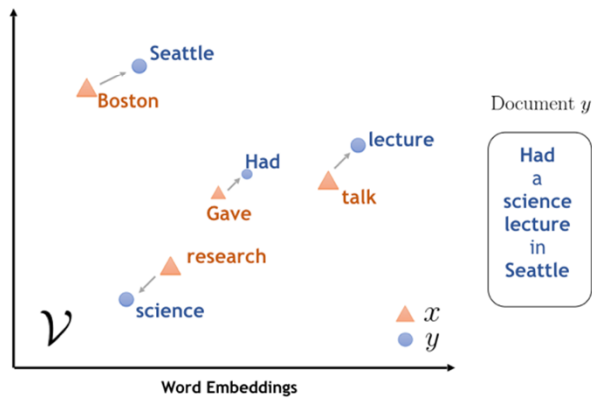


king - man + woman $\approx$ queen



Document x

Gave
a
research
talk
in
Boston



Document y

Had
a
science
lecture
in
Seattle

<http://vku.udn.vn/>

88